

Conecta SUS

Documentacao de Sprints - Projeto

Sistema de Check-in para UBS - Sao Paulo

S1	Fundacao do Projeto
S2	Painel Admin e Correcoes
S3	Melhorias de UX na Fila
S4	Testes de Estresse
S5	Painel TV e Alerta Sonoro

Sprint 1

Fundacao do Projeto

Concluida

Objetivo

Construir a base funcional do sistema de check-in digital para unidades de saude (UBS) de Sao Paulo, com fluxo completo do paciente e verificacao de proximidade geografica.

Estrutura de Arquivos

Arquivo	Responsabilidade
app.py	Pagina inicial e navegacao
database.py	Camada SQLite - fila, tentativas, config
validators.py	Validacao de CPF, Cartao SUS, nome, telefone
shared_state.py	Gerenciamento de estado compartilhado
pages/1_principal_paciente.py	Fluxo principal do paciente (5 etapas)

Fluxo Multi-etapa do Paciente

Etapa	Descricao
1 - Privacidade	Aviso LGPD com consentimento explicito do paciente
2 - Formulario	Nome, CPF/Cartao SUS, telefone, unidade de saude
3 - Localizacao	Verificacao GPS - maximo 10 km da unidade selecionada
4 - Sintomas	Descricao livre dos principais sintomas (min. 5 chars)
5 - Fila	Acompanhamento da posicao em tempo real

Unidades de Saude Cadastradas (10 UBSs)

Unidade	Bairro / Regiao
UBS Valo Velho	Valo Velho
UBS Macedonia	Macedonia
UBS Parque Santo Antonio	Parque Santo Antonio
UBS/AMA Parque Figueira Grande	Figueira Grande
UBS Jardim Maraca	Jardim Maraca
UBS Santa Margarida	Santa Margarida
UBS/AMA Capao Redondo	Capao Redondo
UBS Jardim Germania	Jardim Germania
UBS Sao Bento	Sao Bento
UBS Luar do Sertao	Luar do Sertao

Seguranca e Validacoes

- Anti-spam: bloqueio de CPF apos 3 tentativas em 60 minutos
- Sanitizacao de todos os campos contra injecao HTML/XSS

- Validacao matematica de CPF (dois digitos verificadores)
- Validacao de Cartao SUS (15 digitos)
- Verificacao de duplicidade na fila por CPF + unidade
- Verificacao de proximidade: 10 km no check-in, 1 km na posicao 3

Banco de Dados SQLite

Tabela	Campos principais
fila	id, numero_chamado, nome, cpf_sus, unidade, sintomas, telefone, status
tentativas	id, cpf_sus, timestamp (anti-spam)
config	chave, valor (proximo_numero, ultimo_avanco_fila)

Metricas da Sprint

Indicador	Valor
Arquivos criados	6
Etapas do fluxo	5
Unidades cadastradas	10
Validacoes implementadas	6
Dependencias	3 (streamlit, geopy, streamlit-geolocation)

Sprint 2

Painel Admin e Correcoes de Estabilidade

Concluida

Objetivo

Criar o painel administrativo para gestao da fila pelos funcionarios da unidade e corrigir o bug critico de crash do DOM na geolocalizacao.

Painel Administrativo (pages/2_admin.py)

Dashboard de Estatisticas (5 indicadores)

Indicador	Descricao
Na fila agora	Total de pacientes aguardando atendimento
Atendidos hoje	Total atendido no dia corrente
Total do dia	Soma de atendidos + fila atual
Vagas livres	Capacidade restante (de 100)
Tempo medio	Media de espera calculada dos atendidos

Funcionalidades Admin

- Login com usuario/senha protegido por sessao (admin / sus2025)
- Filtro de fila por unidade de saude
- Botao Chamar proximo paciente por unidade
- Expansor com dados completos de cada paciente
- Remocao individual de pacientes da fila
- Reset total da fila (todas as unidades)
- Reset por unidade individual
- Exportacao de dados em CSV com CPF mascarado (charset UTF-8 BOM)
- Auto-refresh automatico a cada 10 segundos

Correcao Critica: Bug RemoveChild

O componente streamlit_geolocation causava erro fatal de DOM ao ser desmontado:

```
NotFoundError: Falha ao executar "removeChild" em "Node":  
O no a ser removido nao e filho deste no.
```

Causa raiz

O widget era renderizado e desmontado no mesmo ciclo de rerun do Streamlit, causando conflito no ciclo de vida do DOM do React.

Solucao

Criada flag show_geo_widget no session_state. O widget so renderiza apos clique explicito do usuario, e e ocultado antes de qualquer rerun.

Resultado: crash completamente eliminado em todas as etapas do fluxo.

Mettricas da Sprint

Indicador	Valor
Arquivos criados	1 (pages/2_admin.py)
Arquivos modificados	1
Bugs corrigidos	2 ocorrencias do removeChild
Funcionalidades admin	9
Linhas de codigo	~380

Sprint 3

Melhorias de UX na Fila

Concluida

Objetivo

Melhorar a experiencia do paciente na etapa de espera com alertas visuais progressivos por posicao e corrigir dados que persistiam ao reiniciar o check-in.

Sistema de Alertas por Posicao

Posicao	Tipo de alerta	Descricao
1 (proximo)	Card vermelho pulsante	Contagem regressiva 60s + animacao CSS
2	Aviso amarelo	Va agora para a recepcao da unidade
3	Alerta vermelho	Exige estar a 1 km - cancelamento automatico
> 3	Info normal	Posicao + tempo estimado de espera

Posicao 1 - Detalhes

- Card vermelho com animacao CSS @keyframes pulse-border nos bordos
- Numero da senha em destaque (fonte 4.5rem, cor #dc2626)
- Anel de contagem regressiva de 60 segundos
- Atualizacao a cada 5 segundos (em vez dos 15s normais)
- Mensagem: Dirija-se imediatamente ao balcao!

Ajuste de Frequencia de Refresh

Situacao	Antes	Depois
Posicao 1	15 segundos	5 segundos
Demais posicoes	15 segundos	15 segundos

Correcao: Dados Persistindo ao Reiniciar

Problema: o campo telefone nao era apagado ao reiniciar o check-in, aparecendo pre-preenchido no proximo atendimento.

Solucao: adicionadas as chaves 'telefone' e 'tempo_entrada_posicao_1' em todos os 3 pontos de limpeza do session_state.

- Os 3 handlers corrigidos:
1. Fazer novo check-in (apos ser chamado com sucesso)
 2. Fazer novo check-in (apos cancelamento automatico)
 3. Cancelar Check-in (cancelamento manual)

Metricas da Sprint

Indicador	Valor
Arquivos modificados	1
Bugs corrigidos	1 (telefone persistindo)

Novos estados de sessao	1 (tempo_entrada_posicao_1)
Animacoes CSS	2 (pulse-border + countdown-ring)
Classes CSS novas	4

Sprint 4

Testes de Estresse e Validacao

Concluida

Objetivo

Validar a robustez do sistema atraves de testes automatizados cobrindo todos os cenarios extremos: insercao massiva, limite de capacidade, wraparound e anti-spam.

7 Fases de Teste Executadas

Fase	Cenario Testado	Resultado
1	Reset + 80 pacientes	80 numeros unicos, range 1-100
2	Posicao e avanco manual de 10	Ordem e contagem corretas
3	Duplicata de CPF + bloqueio spam	Ambos detectados corretamente
4	Reset + 100 pacientes + 10 extras	Fila bloqueou no 101 paciente
5	Esvaziar 100 + reinserir 20 (wraparound)	Numeracao reinicia sem duplicatas
6	Remocao individual	Posicoes reajustadas corretamente
7	Reset final + estado limpo	Primeiro paciente na posicao 1

Resultado Final

Total OK: 21 verificacoes
Total avisos: 0
Total erros: 0

Nenhum erro critico encontrado em nenhum dos 180+ pacientes simulados.

Comportamento do Sistema Confirmado

Cenario	Comportamento esperado	Status
Fila de 100 pacientes	Numeracao 1-100 unica	OK
Tentativa pos-limite	Retorna None	OK
Wraparound apos reset	Reinicia do 1	OK
CPF duplicado mesma unidade	Bloqueado	OK
3+ tentativas em 60min	Bloqueado por 1h	OK
Remocao individual	Fila reajusta posicoes	OK
Reset total	Estado completamente limpo	OK

Metricas da Sprint

Indicador	Valor
Fases de teste	7
Verificacoes automaticas	21
Pacientes simulados	180+ (80 + 100 + resets + wraparound)

Erros encontrados	0
Tempo de execucao	< 3 segundos

Sprint 5

Painel TV e Alerta Sonoro

Concluida

Objetivo

Criar um painel de exibicao publica para monitores/TVs da recepcao, exibindo a senha chamada e a fila em tempo real, com alerta sonoro automatico a cada novo chamado.

Painel TV (pages/3_painel.py)

Estrutura do Layout

Area	Conteudo
Topbar	Logo + nome do sistema + relógio HH:MM:SS + data
Painel esquerdo	Senha chamada em destaque (10rem) + animacao glow + unidade
Painel direito	Proximas 8 senhas da fila com badges de posicao
Footer	Nome do sistema + LGPD + aviso de atualizacao

Configuracoes Tecnicas

Configuracao	Valor
Layout	Wide (100% da largura da tela)
Tema	Dark - fundo #0f172a, alto contraste para TV
Refresh	A cada 8 segundos automaticamente
Senhas exibidas	8 proximas na fila
Sidebar	Oculto (display:none via CSS)
Animacao	glow-pulse nos bordos da senha chamada

Alerta Sonoro (Web Audio API)

Quando uma nova senha e chamada, o sistema toca automaticamente 3 tons ascendentes diretamente no navegador, sem arquivos de audio externos.

Notas do Beep (3 tons ascendentes)

Tom	Frequencia	Tempo	Duracao
1 (La4)	880 Hz	0.00s	0.18s
2 (Do#5)	1100 Hz	0.22s	0.18s
3 (Mi5)	1320 Hz	0.44s	0.30s

Mecanismo: compara o numero_chamado atual com o do refresh anterior (armazenado em session_state). Se diferente, dispara o audio via st.components.v1.html.

Bug de Renderizacao HTML Resolvido

Problema: o parser Markdown do Streamlit trata linhas com 4+ espacos de indentacao como bloco de codigo, exibindo o HTML como texto literal.

Solucao: todo HTML gerado em variaveis Python foi construido como string concatenada sem indentacao nas linhas, garantindo renderizacao correta pelo Streamlit.

Navegacao Integrada

- Rodape do check-in: link 'Painel TV' abre em nova aba
- Painel Admin: botao 'Painel TV' na barra de acoes

Metricas da Sprint

Indicador	Valor
Arquivos criados	1 (pages/3_painel.py)
Arquivos modificados	2 (check-in + admin)
Linhas de codigo	~160
Dependencias novas	0 (Web Audio API e nativa do navegador)
Bugs resolvidos	1 (HTML com indentacao excessiva)